Format for describing dependencies of source files

Ben Boeckel, Brad King
<ben.boeckel@kitware.com, brad.king@kitware.com>
version P1689R3, 2020-12-09

Table of Contents

Document number	ISO/IEC/JTC1/SC22/WG21/P1689R3
Date	2020-12-09
Reply-to	Ben Boeckel, Brad King, ben.boeckel@kitware.com, brad.king@kitware.com
Audience	EWG (Evolution), SG15 (Tooling)

1. Abstract

When building C++ source code, build tools need to discover dependencies of source files based on their contents. This occurs during the build so that file contents can change without build tools having to reinitialize and so the dependencies of source file generated during the build are correct. With the advent of modules in [P1103R3], there are now ordering requirements among compilation rules. These also need to be discovered during the build. This paper specifies a format for communicating this information to build tools.

2. Changes

2.1. R3 (Dec 2020 mailing)

Changed:

- `work-directory` is now represented per-rule rather than as a top-level entry.
- arbitrary binary data format storage has been removed. This was deemed as too complicated for the benefits gained at this time. If experience shows that the generalizations are needed, the binary representation can be revisited in the future.
- the example filenames are now consistent with each other.

2.2. R2 (pre-Prague)

Added:

- more background information (motivation and assumptions)
- validity of source entries depends on uniqueness in the outputs, not the inputs.
- `input` is now an array, `inputs`. This is to support representation of unity builds where multiple input sources are compiled at once into a single set of outputs.
- `sources` is renamed to `rules`. There is not necessarily a 1-to-1 correlation with source files with compilation rules.
- full example output for a C++ source
- uniformly use "property" instead of "key" for JSON fields.

2.3. R1 (post-Cologne)

The following changes have been made in response to feedback in SG15:

- rename keys to be more "noun-like" or for clarity including:
- `readable` → `readable-name`
- `logical` → `logical-name`
- `filepath` → `compiled-module-path`
- remove `future-link` (no known use case)
- remove `%`-encoding for filepaths
- remove top-level `extensions` key (still possible, just use `_` keys)
- require vendor prefixes for extensions
- add an optional `source-path` key to `depinfo` objects

The following changes have been made in response to feedback in SG16:

- change the name of the "data" key to "code-units"
- mention normalization for JSON encoders and decoders

2.4. R0 (Initial)

Description of the format and its semantics.

3. Introduction

This paper describes a format designed primarily to communicate dependencies of source files during the building of C++ source code. While other uses may exist, the primary goal is correct compilation of C++ sources. The tool which generates this format is referred to as a "dependency scanning tool" in this paper.

The contents of this format includes:

- the dependencies of running the dependency scanning tool itself;
- the resources that will be required to exist when the scanned translation unit is compiled; and
- the resources that will be provided when the scanned translation unit is compiled.

This information is sufficient to allow a build tool to order compilation rules to get a valid build in the presence of C++ modules.

4. Motivation

Before C++ modules, the only kinds of dependencies on files that a build system would care about could be determined during the execution of that rule. This is because each compilation was independent of other compilation rules. However, with modules, compilation rules can now depend on each other and they must be executed in order. Build tools need to be able to extract this information from source files before compiling them due to this new ordering requirement.

Incidentally, this is exactly analogous to the problem that Fortran build systems has with Fortran modules. To that end, this format is explicitly not specific to C++ and is intended for use within the Fortran ecosystem in the future. Terminology specific to C++ is avoided in this format to avoid any indications that it is C++-specific.

4.1. Why Makefile snippets don’t work

Historically, dependency information of a build rule has been handled by Makefile snippets. An example of this is:

```
output: input_a input_b input_c
```

This states that the artifact of the build rule is `output` and files `input_a`, `input_b`, and `input_c` were read during its creation. This allows the build system to know that if any of the listed `input_*` files changes, the rule for `output` needs to be brought up-to-date as well.

This works decently well for the kinds of dependencies that have occurred in C++ to date, namely header includes. This is because these dependencies can be discovered while executing the rule associated with `output`.

The issue that arises with the Makefile design is that modules are a new kind of dependency that cannot be represented in declarative Makefile syntax. For example, GCC outputs variable modifications (`CXX_MODULES+=…`) into these snippets which is commonly not supported by the consuming tools. In addition, because these dependencies must be discovered **before** the compilation rule is executed, there would need to be one rule that writes dependency information for another.

As an example of the restrictions placed on these Makefile snippets, the `ninja` [ninja] build tool requires that `output` be the same for the rule which wrote out the dependency snippet and that no other outputs are mentioned. No other Makefile syntax is supported (variables, adding rules, special variables, macro expansions, etc.). This is because `ninja` is reading these for just the dependency information.

5. Assumptions

This format assumes the following things about the environment in which it is used: uniformity of the environment between creation and usage; only used within one build of a project; it does not apply to different configurations of a build (since dependencies may vary with the target platform or build settings such as whether it is debug or not).

It is generally assumed that the environment in which a file of this format is created is the same as the environments in which it will be read and ultimately used during the actual compilation. However, build systems may have different strategies for executing rules and when this is the case, it is assumed that the build system itself knows how to translate between the environments it sets up for each rule. For example, a build system which distributes the builds across multiple machines (whether over a network or using containerization) should know how to translate between the environment set up for one execution and another execution.

Environments can have many knobs which change fundamental behaviors of the system. A non-exhaustive list includes:

- mount layout (particularly of the input and output absolute paths)
- encoding (active code page, locale)
- effective permissions (process user and group, security modules, anti-virus)

The first two can be translated between different rules in a straightforward way. For example, if one rule is executed in a `/chroot/exec1` prefix while another is under `/chroot/exec2`, it is assumed that the build system constructed those environments and knows that paths underneath those prefixes should be rerooted for another execution rule to get its paths correct. Encoding differences can be converted between using either system APIs or libraries which handle encodings. If there are permission differences between the scanner and the compiler, it is hard to imagine how a build tool would be able to translate the file effectively.

6. Format

The format uses JSON [ECMA-404] as a base for encoding its information. This is suitable because it is structured (versus a plain-text format), parsers for JSON are readily available (versus candidates with a custom structural format), and the format is simple to implement (versus candidates such as YAML or TOML) which will allow for easy adoption.

JSON specifies that documents are Unicode. However, due to the way filepaths are represented in this format, it is further constrained to be a valid UTF-8 sequence.

6.1. Schema

For the information provided by the format, the following JSON Schema [JSON-Schema] may be used.

▶ JSON Schema for the format

6.2. Storing binary data

This format uses UTF-8 as a communication channel between a dependency scanning tool and a build tool, but filepath encodings are specific to the platform which means considerations for paths containing non-UTF-8 sequences must be made. However, the most common uses of paths and filenames are either valid UTF-8 sequences or may be unambiguously represented using UTF-8 (e.g., a platform using UTF-16 for its path APIs has a valid UTF-8 encoding), so requiring excessive obfuscation in all cases is unnecessary.

After discussion with stakeholders, complicating the format for corner cases of filepaths which do not have unambiguous UTF-8 representations is an unnecessary complication at the moment. Future versions of the format may have a way to unambiguously transmit filepaths that are not Unicode-unambiguous or not valid Unicode if the need arises..

There are some use cases (though rare) which cannot be handled without a way to represent arbitrary paths. These include (but are not limited to):

- Windows paths with unpaired surrogate half codepoints (for which there is no valid UTF-8 representation).
- Encodings historically used for East Asian languages including Big-5, SHIFT-JIS, and others. There are characters in these encodings which share a Unicode representation, so there is no lossless way to use UTF-8 strings as a transport for these paths.

These restrictions have been deemed to not be important enough to support at this time in the general format. Note that many build tools already have restrictions in characters due to implementation details. For example, Makefiles have trouble representing paths ending with a `\\` character and CMake has issues with paths containing its list separator, the semicolon.

6.3. Filepaths

Filepaths may either be relative or absolute. To this end, the dependency scanning tool must output its working directory in the `work-directory` property for each rule. The build tool may then construct the absolute paths as necessary.

For concrete examples where absolute paths may not be suitable:

- A distributed build may perform the compilation in a different directory on another machine than the host machine is using.
- A build tool uses a chroot for each command it invokes.[1]

6.4. Rule items

The `rules` array allows for the dependency information of multiple rules to be specified in a single file.

The only restriction on the contents of the collective set of `rules` objects is that the set of all `outputs` in each object and `future-compile` object must be unique (after translation into the

appropriate environment). This is because if they are not unique, there are outputs which have multiple rules that write to them, which is, in general, undefined behavior in build tools.

6.5. Dependency information

Each rule represented in the `rules` array is a JSON object which has a single required property, `inputs`. The value of this property is an array of `datablock` entries representing a set of filepaths that are read directly by the execution of this rule. A rule must have at least one input because otherwise the rule is idempotent and never needs dependency information to be discovered. Two optional properties exist to indicate the dependencies of the execution of the dependency scanning tool itself: the `outputs` array and the `depends` array. The `depends` value is an array in which each element is a filepath for files that affect the results of the run. For C++, this will generally be paths due to `#include`, but other mechanisms may be in effect (e.g., the proposed `#embed` directive discussed in [P1040R6] and [P1967R2]).

6.6. Future dependency information

The core of this specification is the `future-compile` property on a `rules` object. `future-compile` objects have three optional properties, `outputs`, `provides`, and `requires`.

The `outputs` array contains filepaths which will be written to when the source is compiled (e.g., object files or debugging sidecar files). The `provides` and `requires` arrays contain names of modules. The `provides` array is for modules that the inputs will produce and the `requires` array is for modules that the inputs require. Each item of these arrays is a JSON object with one required property, `logical-name`, and two optional properties: `compiled-module-path` and `source-path`. All of these property's values are filepaths. The `logical-name` value is what build tools must use to discover the ordering among translation unit compilations. In C++, this will generally be the name of the module (including its partition, if any) as included in the source. The `compiled-module-path` should be provided only if the location of the module's artifact is known when the dependency information is discovered. The `source-path` is the path to the main source of the module. This is intended to be used to communicate the location of a header for a header-unit import or a module interface unit for a C++20 module when it is known.

Example source entry with `future-compile` information

```
{
  "inputs": [
    "path/to/input.cxx"
  ],
  "future-compile": [
    "outputs": [
      "path/to/output.o"
    ],
    "provides": [
      {
        "compiled-module-path": "exported.bmi",
        "logical-name": "exported"
      }
    ],
    "requires": [
      {
        "logical-name": "imported"
      }
```

```
    ]
  ]
}
```

6.7. Extensions

Vendor extensions may be added to the format using properties prefixed with an underscore (_) followed by a vendor-specific string followed by another underscore. None of these may be used to store semantically relevant information required to execute a correct build. Consumers must be able to ignore all _-prefixed properties and not suffer any loss of essential functionality.

Example source entry with extended information

```
{
  "input": "path/to/input",
  "_VENDOR_extension": true
}
```

7. Versioning

There are two properties with integer values in the top-level JSON object of the format: `version` and `revision`. The `version` property is required and if `revision` is not provided, it can be assumed to be `0`. These indicate the version of the information available in the format itself and what features may be used. Tools creating this format should have a way to create older versions of the format to support consumers that do not support newer format versions.

The `version` integer is incremented when semantic information required for a correct build is different than the previous version. When the `version` is incremented, the `revision` integer is reset to `0`.

The `revision` integer is incremented when the semantic information of the format is the same as the previous revision of the same version, but it may include additionally specified information or use an additionally specified format for the same information. For example, adding a `modification_time` or `input_hash` field may be helpful in some cases, but is not required to understand the dependency information. Such an addition would cause an increment of the `revision` value.

The version specified in this document is:

Version fields for this specification

```
{
  "version": 1,
  "revision": 0
}
```

8. Full example

Given this reduced source file `module.cpp`:

Reduced example C++ module source

```
export module my.module;

import other.module;
import <header>;

#include "config.h"
```

a full output for scanning this source could be:

Example dependency output

```
{
  "version": 1,
  "revision": 0,
  "rules": [
    "work-directory": "/scanner/working/dir",
    "inputs": [
      "my.module.cpp"
    ],
    "outputs": [
      "depinfo.json"
    ],
    "depends": [
      "/system/include/path/header",
      "include/path/config.h"
    ],
    "future-compile": {
      "outputs": [
        "my.module.cpp.o",
        "my_module.bmi"
      ],
      "provides": [
        {
          "logical-name": "my.module",
          "source-path": "my.module.cpp",
          "compiled-module-path": "my_module.bmi"
        }
      ],
      "requires": [
        {
          "logical-name": "other.module"
        }
        {
          "logical-name": "<header>",
          "source-path": "/system/include/path/header",
        }
      ]
    }
  ]
}
```

9. References

- [ECMA-404] The JSON Data Interchange Syntax. http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf.
- [JSON-Schema] Austin Wright and Henry Andrews. JSON Schema: A Media Type for Describing JSON Documents. https://tools.ietf.org/html/draft-handrews-json-schema-01.
- [ninja] Ninja, a small build system with a focus on speed. https://ninja-build.org/.
- [P1040R6] JeanHeyd Meneide. std::embed and #depend. http://open-std.org/JTC1/SC22/WG21/docs/papers/2020/p1040r6.html.
- [P1103R3] Richard Smith. Merging Modules. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1103r3.pdf.
- [P1967R2] JeanHeyd Meneide. #embed - a simple, scannable preprocessor-based resource acquisition method. http://open-std.org/JTC1/SC22/WG21/docs/papers/2020/p1967r2.pdf.
- [RFC3629] Francois Yergeau. UTF-8, a transformation format of ISO 10646. https://tools.ietf.org/html/rfc3629.
- [Unicode-12] Unicode Consortium. Unicode 12.0.0. https://www.unicode.org/versions/Unicode12.0.0/.

1. Concretely, the Tup build tool can execute compile rules inside of individual FUSE chroots where absolute paths are meaningless outside of that context. In this case, Tup would know how to translate between the producer's and consumer's different chroot paths.